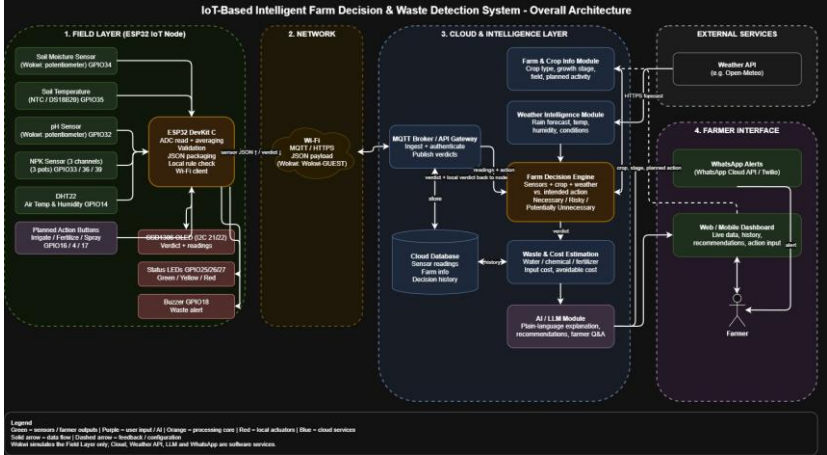


FarmGuard: One-Page Technical Progress Report

IoTriX 2.0 Semi-Final | Track A: Embedded IoT System | South Eastern University of Sri Lanka | Team: T4D |

<https://github.com/BinMushan/FarmGuard>

Problem	Smart farming systems show sensor values but do not tell a farmer whether a planned action (irrigation, fertilizer, spraying) is actually needed. Farmers often act out of habit or with incomplete information, which wastes water, fertilizer and money and can harm the crop. Target users are small and medium-scale farmers.
Proposed Solution	FarmGuard is an ESP32 field node that reads soil and air sensors and lets the farmer select a planned action with a button. A rule-based engine classifies it as Necessary (green), Risky (yellow) or Potentially Unnecessary (red + buzzer), shows the verdict locally on an OLED and sends data over Wi-Fi to a cloud engine that adds the weather forecast and crop stage, estimates the avoidable cost, and sends alerts through a dashboard and WhatsApp. An LLM only explains the result; it does not decide.
Architecture	 <i>Field layer (ESP32 node), network (Wi-Fi, MQTT / HTTPS, JSON), cloud and intelligence layer (broker, database, decision engine, weather, cost estimate, LLM) and farmer interface (dashboard, WhatsApp).</i>
Current Progress	- System architecture designed (all four layers and data flow). - Field node circuit built and wired in Wokwi: ESP32 DevKit C with soil moisture (GPIO34), pH (32), N (33), P (36), K (39), soil temperature NTC (35), DHT22 (14), SSD1306 OLED (I2C 21/22), 3 action buttons (16, 4, 17), 3 status LEDs (25, 26, 27) and a buzzer (18). All analog sensors are on ADC1 so they stay usable with Wi-Fi on.
Technology Stack	Hardware: ESP32, soil moisture, NTC, pH, NPK sensors, DHT22, OLED, LEDs, buzzer, buttons. Firmware: Arduino (C++). Protocols: Wi-Fi, MQTT / HTTPS, JSON. Cloud: broker / API gateway, backend, database, rule engine, web dashboard AgriMain AI. Services: weather API, LLM API, WhatsApp API. Tools: Wokwi, GitHub.
Testing and Validation	No measured results yet. Simulation alone is not our proof, so every test is run in Wokwi and then on real hardware, and the two are compared. Simulation under different conditions: 9 scenarios (dry soil, in-range moisture, saturated soil, N/P/K in range or low, rain forecast, bad sensor value, Wi-Fi off) checked against expected verdicts (green / yellow / red + buzzer). Real-world validation: the same firmware logic on a physical ESP32 with real sensors; readings checked against reference measurements (weight-based soil moisture, reference thermometer, pH 4 and 7 buffers); real physical test: water dry soil and watch the verdict change. Validity measure: verdict agreement between Wokwi and hardware for the same inputs, reading error, and response to change. Mismatches will be reported. Full plan in the proposal (Section 15).
Limitations	Circuit is currently simulated (potentiometers replace moisture, pH and NPK sensors), so real-hardware validation is still required; real NPK sensors usually need RS485. Low-cost NPK and pH sensors drift (highest technical risk). One sensor point per field. Thresholds not yet validated with an expert. Not field-hardened. Weather, LLM and WhatsApp services can fail, so the rule-based verdict is always shown without them.
Next Step	Build the physical prototype with real sensors, finish and test the firmware (including offline buffering), connect the cloud engine, dashboard, WhatsApp and LLM end to end, complete the Wokwi scenario tests and the real-hardware validation, calibrate sensors, validate thresholds, and report results on real soil.